

FREE HANDS-ON COURSE · EXPLAINED VISUALLY

Kubernetes, from zero to practical: **every command tested**

29 lessons on a cluster on your own computer. Deploy it, inspect it, break it, fix it.

29

hands-on lessons

14

broken clusters to fix

750

code blocks tested in CI

13 + 1

labs + a capstone

THE REAL PROBLEM

**Tutorials end at `kubectl apply`.
Work starts with these:**

```
pod/writer 0/1 Pending 0 5s
worker-5b6dd64668-nkhvt 0/1 CrashLoopBackOff 1 (2s ago)
backend-7c96cd4d65-9gzpz learning-app/backend:3.0.0
ImagePullBackOff
import 0/1 OOMKilled 0 5s
Readiness probe failed: HTTP probe failed with statuscode: 404
Selector: app=back-end
Endpoints:
wget: bad address 'backend'
```

Real outputs from the course's troubleshooting lab. Each one is reproduced, investigated and fixed.

THE METHOD

Every concept, the same path

Theory

Visual

YAML

kubectl

Deploy

Inspect

Experiment

Break it

Troubleshoot

Fix it

Every lesson explains its YAML field by field, has a lab with the expected result, a failure made on purpose with its real events, a challenge with a hidden solution, and "where you use this in a DevOps job".

You declare. Kubernetes keeps it true.

Deployment

replicas: 3
image: backend:2.0.0

ReplicaSet

"3 wanted, 2 running"
→ create one

Pods

3 running again
on any healthy node

Delete a Pod and it comes back. Update the image and old Pods are replaced one by one; a bad image stops the rollout while the old version keeps serving, and **kubectl rollout undo** brings it back.

One stable name for Pods that come and go

```
$ kubectl describe service backend-service
Selector:  app=back-end
Endpoints:
$ kubectl get pods --show-labels
backend-...-5jt79    1/1    Running    app=backend
```

A Service finds its Pods by their labels. One typo in the selector and it has no endpoints: every Pod is healthy, and every request fails.

Only the allowed paths, only the allowed people

ingress

→ frontend :8080

frontend

→ backend :8080

backend

→ database :5432

```
$ kubectl auth can-i list pods --as developer
yes
$ kubectl auth can-i delete pods --as developer
no
```

Default-deny policies enforced by Calico (the frontend can never reach the database), and a read-only Role for people.

THINGS THE TESTS TAUGHT ME

Surprises worth knowing

A PVC still used by a finished Job's Pod waits in **Terminating** forever

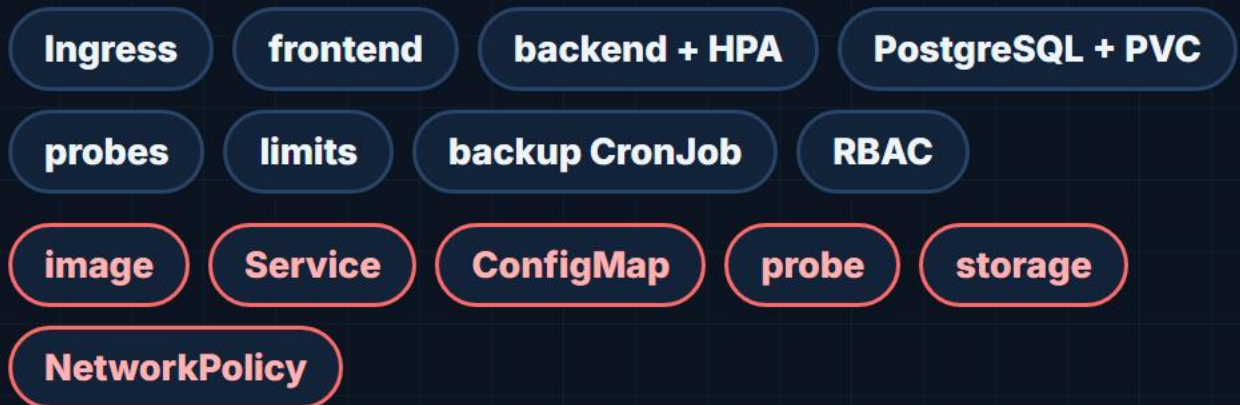
runAsNonRoot fails when the image's user is a name, not a number

A probe's default timeout is **1s**: a database check can fail it

A Minikube node reports the **host's** CPUs, not the ones you gave it

THE CAPSTONE

A production-style application, then six breaks



Each break: the user's symptom, the object that explains it, the root cause, the fix from the reviewed files, and a verification as the user.

WHAT IS INSIDE

Everything to learn it, and prove it

13

hands-on labs

19

challenges, solutions hidden

17

videos, full + silent

117

page study guide PDF

Plus a Helm chart with values per environment, 13 diagrams, a glossary, 22 interview questions and a final knowledge checklist.

EVERY COMMAND TESTED

The lessons are the test suite

```
←!— test: contains=Readiness probe failed; retry=20 →  
```bash  
kubectl get events -n trouble-06 --field-selector reason=Unhealthy
...
```
```

A runner executes every block as a learner types it, on a real Minikube cluster, resets the cluster between lessons and writes the real output back. GitHub Actions does the same on a fresh cluster.

FREE AND LOCAL

minikube start



the course



all my projects

Which Kubernetes error cost you the most time?